

claude-windows / 7fe4ef45-d30a-448e-98eb-4da4fba244f1

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fe4ef45-d30a-448e-98eb-4da4fba244f1\subagents\workflows\wf\_b531eaeef-39c\agent-a6ecc71be15cc6196.jsonl`
- SHA-256: `eeae67aaefa6557a529382a695aa7e6cbdf94577270182ff0ec0020bec1aba9`
- Source modified: `2026-06-19T16:58:59+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf\_b531eaeef-39c`
- session_id: `7fe4ef45-d30a-448e-98eb-4da4fba244f1`

Transcript

1. user (2026-06-19T16:57:07.034Z)

You are verifying claims for a design doc; the doc must only assert what the code supports. Verify: (a) WHERE the current single-target tracker actually lives ? is build_tracker importing from the WASB-SBDT repo's 'trackers' package (runs only inside WSL), per wasb_infer.py imports? (b) Therefore could a REPLACEMENT multi-target linker run as PURE PYTHON on Windows over the cached det_raw.jsonl / trajectory CSV WITHOUT WSL/GPU? (c) Licensing: confirm WASB-SBDT code is MIT and that a custom tracker authored in THIS repo is our own IP (consistent with the MIT/Apache/BSD-only guardrail in CLAUDE.md). Search the repo + docs for license claims. Cite evidence. VERIFIED/REFUTED/PARTIAL.

Read the actual files (do not guess). Return the structured verdict.

2. assistant (2026-06-19T16:57:09.934Z)

I'll verify these claims by reading the actual code. Let me start by finding the relevant files.

3. user (2026-06-19T16:57:10.602Z)

```
.claude\worktrees\bhi-execpolicy\backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\wasb_infer.py
backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\tier1-windowing\backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\w2-safeguard\backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\hardchop\backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\boxhill17\backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\pr-stitch\backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\gap-doc\backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\resweep\backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\recdoc\backend\pipeline\detectors\wasb_infer.py
```

4. user (2026-06-19T16:57:11.030Z)

```
Found 2 files
docs\CODE_MAP.md
backend\pipeline\detectors\wasb_infer.py
```

5. user (2026-06-19T16:57:11.869Z)

```
backend\pipeline\segmenters\base.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\segmenters\__init__.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py
```

```
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\gemini.py
```

6. assistant (2026-06-19T16:57:14.142Z)

Let me read the key files.

7. user (2026-06-19T16:57:15.311Z)

```
1      """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3      WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4      own building blocks (ImageDataset transform + detector + online tracker) to run on
5      any folder of extracted frames and emit a trajectory CSV in the exact shape the
6      indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8          Frame,Visibility,X,Y
9
10     It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11     imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12     (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14     Resumable across reboots
15     -----
16     The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17     the slow, expensive part; the CPU tracker pass that turns detections into a
18     trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19     resume mid-stream). We exploit that split:
20
21     * The detector pass writes its per-window raw outputs incrementally to
22       ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23       records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24       most ``--flush-every`` windows of GPU work instead of the whole pass.
25     * On restart we replay the cached windows, skip the DataLoader ahead to the first
26       un-cached window, and continue. Once every window is cached, the detector pass
27       is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28       the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29       not the GPU hour.
30
31     The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32     to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33     weights, sport, window size, or frame count change.
34
35     Usage (in WSL, from ~/models/WASB-SBDT/src):
36         python wasb_infer.py --frames_dir /path/frames --weights
37         ../pretrained_weights/wasb_badminton_best.pth.tar \
38         --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
39         every 1000] [--fresh]
40
41     """
42     import os
43     import os.path as osp
44     import csv
45     import glob
46     import json
47     import argparse
48     import logging
49     from collections import defaultdict
50     from contextlib import contextmanager
51
52     logger = logging.getLogger(__name__)
53
54     CACHE_VERSION = 1
55     _DET_FILE = "det_raw.jsonl"
56     _MANIFEST_FILE = "manifest.json"
```

```

54
55
56 def _build_cfg(weights: str, sport: str):
57     """Compose the WASB eval config via Hydra, overriding model/weights/device."""
58     from hydra import compose, initialize
59     with initialize(config_path="configs", version_base=None):
60         cfg = compose(config_name="eval", overrides=[
61             f"dataset={sport}",
62             "model=wasb",
63             f"detector.model_path={weights}",
64             "runner.device=cuda",
65             "runner.gpus=[0]",          # this box has a single GPU; default config assumes
more
66         ])
67     return cfg
68
69
70 def _frame_id(path: str) -> int:
71     """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
180)."""
72     stem = osp.splitext(osp.basename(path))[0]
73     digits = "".join(ch for ch in stem if ch.isdigit())
74     return int(digits) if digits else -1
75
76
77 # ----- #
78 # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
79 # ----- #
80 def _cache_paths(cache_dir: str):
81     return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
82
83
84 def default_cache_dir(out_csv: str) -> str:
85     """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
86     d = osp.dirname(osp.abspath(out_csv))
87     stem = osp.splitext(osp.basename(out_csv))[0]
88     return osp.join(d, stem + "_wasbcache")
89
90
91 def _atomic_write_text(path: str, text: str) -> None:
92     """Write then rename so a crash mid-write can't leave a half-written file."""
93     tmp = path + ".tmp"
94     with open(tmp, "w") as f:
95         f.write(text)
96         f.flush()
97         os.fsync(f.fileno())
98     os.replace(tmp, path)
99
100
101 def _det_plain(det) -> list:
102     """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
list."""
103     xy = det["xy"]
104     return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
105
106
107 def _dets_to_tracker(plain_list):
108     """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
array,
109     the tracker does vector math / np.linalg.norm on it)."""
110     import numpy as np
111     return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
112             for p in plain_list]
113
114

```

```

115 def write_manifest(cache_dir: str, manifest: dict) -> None:
116     _, mpath = _cache_paths(cache_dir)
117     _atomic_write_text(mpath, json.dumps(manifest, indent=2))
118
119
120 def read_manifest(cache_dir: str):
121     _, mpath = _cache_paths(cache_dir)
122     if not osp.exists(mpath):
123         return None
124     try:
125         with open(mpath) as f:
126             return json.load(f)
127     except (json.JSONDecodeError, OSError):
128         return None
129
130
131 def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
132                         frames_in: int, frames_total: int) -> bool:
133     """A cache is reusable only if the run that produced it matches this run's
134     inputs."""
135     if not manifest or manifest.get("version") != CACHE_VERSION:
136         return False
137     return (
138         manifest.get("frames_dir") == osp.abspath(frames_dir)
139         and manifest.get("weights") == weights
140         and manifest.get("sport") == sport
141         and manifest.get("frames_in") == frames_in
142         and manifest.get("frames_total") == frames_total
143     )
144
145 def reset_cache(cache_dir: str) -> None:
146     """Drop any existing cache so the next run starts clean."""
147     det_jsonl, mpath = _cache_paths(cache_dir)
148     for p in (det_jsonl, mpath):
149         if osp.exists(p):
150             os.remove(p)
151
152
153 def load_and_clean_cache(cache_dir: str):
154     """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
155     rebuild the per-frame detection accumulation, and rewrite the file to exactly that
156     prefix so a trailing half-written line from a crash can't corrupt future appends.
157
158     Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
159     [x, y, score, scale] candidates (accumulated across every window the frame is in,
160     in window order ? identical to a non-resumed run).
161     """
162     det_jsonl, _ = _cache_paths(cache_dir)
163     det_by_fid = defaultdict(list)
164     valid: list = []
165     if osp.exists(det_jsonl):
166         with open(det_jsonl, "r") as f:
167             for line in f:
168                 s = line.strip()
169                 if not s:
170                     continue
171                 try:
172                     obj = json.loads(s)
173                 except json.JSONDecodeError:
174                     break # truncated trailing line (crash mid-
flush)
175                 if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
176                     break
177                 valid.append(s)

```

```

178         for fid, plain in obj["f"]:
179             det_by_fid[int(fid)].extend([list(p) for p in plain])
180             # Guarantee a clean append boundary by rewriting only the good prefix.
181             _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
182         return len(valid), det_by_fid
183
184
185 def _append_windows(fh, objs) -> None:
186     """Append window records as JSONL and durably flush (bounded loss on crash)."""
187     for o in objs:
188         fh.write(json.dumps(o, separators=(",", ":")) + "\n")
189     fh.flush()
190     os.fsync(fh.fileno())
191
192
193 def _atomic_write_trajectory(out_csv: str, rows) -> None:
194     os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
195     tmp = out_csv + ".tmp"
196     with open(tmp, "w", newline="") as f:
197         w = csv.writer(f)
198         w.writerow(["Frame", "Visibility", "X", "Y"])
199         w.writerows(rows)
200         f.flush()
201         os.fsync(f.fileno())
202     os.replace(tmp, out_csv)
203
204
205 # ----- #
206 # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
207 # ----- #
208 _STREAM_FRAME_FMT = "{:08d}.png"
209
210
211 def synthetic_frame_path(index: int) -> str:
212     """Stable, index-encoding frame name used by the streaming path.
213
214     It mirrors the on-disk names ``extract_frames`` writes (``{:08d}.png``) so the
215     downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
216     frames came from disk or from the in-memory stream. The streaming image loader
217     inverts this name back to an index to fetch the decoded frame.
218     """
219     return _STREAM_FRAME_FMT.format(int(index))
220
221
222 def build_windows(frames: list, frames_in: int):
223     """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
224     + checkpoint). Pure list math, identical for the disk and streaming paths.
225
226     Returns a list of windows, each a list of ``frames_in`` consecutive paths
227     (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
228     caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
229     ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
230     """
231     return [frames[i:i + frames_in] for i in range(len(frames) - frames_in + 1)]
232
233
234 class SequentialFrameStore:
235     """In-memory sliding buffer over a forward-only frame reader, sized for the
236     detector's sliding-window access pattern (peak O(window) frames, not O(video)).
237
238     The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
239     ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
240     in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
241     start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
242     boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so

```

```

243     far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
244     earliest index any not-yet-finished window can still ask for) and decode each source
245     frame exactly once via the forward-only ``reader(index)``.
246     """
247
248     def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
249         self._reader = reader
250         self._total = int(total)
251         self._window = max(1, int(window))
252         self._buf: dict = {}
253         # On a resumed run the detector's first needed frame is `start_index`; begin
254         # pulling there so the reader can skip (seek past) the already-cached prefix
255         # instead of decoding 0..start_index-1 just to throw them away.
256         self._next = int(start_index)      # next index not yet pulled from the reader
257         self._max_seen = int(start_index) - 1
258         self._floor = int(start_index)     # lowest index we still keep (never evict
below)
259
260     def get(self, index: int):
261         index = int(index)
262         if index < self._floor:
263             raise KeyError(
264                 f"frame {index} already evicted (below floor {self._floor}); the access
"
265                 f"pattern must stay within a window of the highest frame seen")
266         if index >= self._total:
267             raise KeyError(f"frame {index} out of range (total {self._total})")
268         # Pull forward from the reader until the requested index is buffered.
269         while self._next <= index:
270             self._buf[self._next] = self._reader(self._next)
271             self._next += 1
272         if index > self._max_seen:
273             self._max_seen = index
274         # Evict frames older than the earliest a still-running window could re-request:
275         # once we've seen index `m`, no future window starts before `m - (window-1)`.
276         new_floor = max(self._floor, self._max_seen - (self._window - 1))
277         for k in range(self._floor, new_floor):
278             self._buf.pop(k, None)
279         self._floor = new_floor
280         return self._buf[index]
281
282
283     def _index_from_synthetic_path(path: str) -> int:
284         """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
285
286         Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
287         in lockstep with the on-disk naming scheme.
288         """
289         return _frame_id(path)
290
291
292     def _bgr_to_pil_rgb(frame_bgr):
293         """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
EXACT
294         PIL RGB image WASB's disk path produces.
295
296         The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
Image.open(PNG).convert('RGB')``;
297         because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
COLOR_BGR2RGB)``
298         bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
streamed
299         frame indistinguishable from the on-disk one to everything downstream.
300         """
301         import cv2

```

```

302     from PIL import Image
303     return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
304
305
306     def _make_streamed_read_image(store, real_read_image):
307         """Build the ``read_image`` replacement used during a streaming detector pass.
308
309         A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
310 image,
311 matching ``read_image``'s real return type); any path that doesn't parse to an index
312 falls
313 through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
314 testable
315 without the WSL-only ``dataloaders`` package.
316 """
317     def _read_image(path, *args, **kwargs):
318         idx = _index_from_synthetic_path(path)
319         if idx < 0:
320             return real_read_image(path, *args, **kwargs)
321         return _bgr_to_pil_rgb(store.get(idx))
322     return _read_image
323
324     @contextmanager
325     def _streaming_read_image_patch(frame_loader, total: int, window: int = 1, start_index:
326 int = 0):
327         """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
328 decoded
329 frames during the detector pass, byte-for-byte as it would over real PNGs.
330
331 WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
332 ``cv2.imread``. ``read_image`` (``utils.utils``) does
333 ``Image.open(path).convert('RGB')``,
334 returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
335 feed it a
336 PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
337 byte-identical to the disk round-trip, and the shim never touches the filesystem so
338 the
339 ``osp.exists`` guard is sidestepped for synthetic stream paths.
340
341 We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
342 actually
343 calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
344 in that
345 module's namespace; patching ``utils.read_image`` would NOT rebind the already-
346 imported
347 reference).
348
349 ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
350 wrapped
351 in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
352 on a
353 resume we start at ``start_index`` so the decoder seeks past already-cached windows.
354 The
355 original reader is restored on exit.
356 """
357     from dataloaders import dataset_loader as _dl
358     store = SequentialFrameStore(frame_loader, total, window=window,
359 start_index=start_index)
360     real_read_image = _dl.read_image
361     # Rebind read_image in the consuming module for the duration of the detector pass;
362     # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
363 B010-clean.)
364     _dl.read_image = _make_streamed_read_image(store, real_read_image) # type:
365 ignore[assignment]

```

```

350         try:
351             yield store
352         finally:
353             _dl.read_image = real_read_image # type: ignore[assignment]
354
355
356     # ----- #
357     # Inference
358     # ----- #
359     def run(frames, weights: str, out_csv: str, sport: str = "badminton",
360             limit: int = 0, batch_size: int = 8, num_workers: int = 4,
361             log_every_batches: int = 50, cache_dir: "str | None" = None,
362             flush_every: int = 1000, fresh: bool = False,
363             frame_loader=None, source_key: "str | None" = None) -> str:
364         """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
365
366         Two equivalent ways to supply pixels (output is bit-identical between them):
367
368         * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
369           globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
370           ``cv2.imread``.
371         * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
372           frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
373           pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
374           reader, which returns a PIL RGB image) over the DataLoader pass so every other
375           line of
376           ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
377           identical
378           to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
379           ``cvtColor(bgr, BGR2RGB)``), verified byte-identical). Streaming forces
380           ``num_workers=0``
381           (the in-process shim + buffer can't cross worker processes).
382
383         ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
384         frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
385         after reboot still matches.
386         """
387         import time
388         import torch
389         from torch.utils.data import DataLoader
390         from detectors import build_detector
391         from trackers import build_tracker
392         from dataloaders import build_img_transforms, build_seq_transforms
393         from dataloaders.dataset_loader import ImageDataset
394         from utils import Center
395
396         streaming = frame_loader is not None
397
398         cfg = _build_cfg(weights, sport)
399         frames_in = int(cfg["model"]["frames_in"])
400         input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
401         output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
402
403         if streaming:
404             # ``frames`` is already the ordered list of (synthetic) frame paths.
405             if not isinstance(frames, (list, tuple)):
406                 raise SystemExit("streaming mode requires ``frames`` to be a list of frame
407                 paths")
408             frames = list(frames)
409             frames_dir = source_key or "stream"
410         else:
411             frames_dir = frames
412             frames = sorted(glob.glob(osp.join(frames_dir, "*.png"))) +
413                 glob.glob(osp.join(frames_dir, "*.jpg"))
414             if limit:

```

```

409         frames = frames[:limit]
410     if len(frames) < frames_in:
411         where = frames_dir if not streaming else "video stream"
412         raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
413
414     # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
415     samples = []
416     for window in build_windows(frames, frames_in):
417         annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
for p in window]
418         samples.append({"frames": window, "annos": annos})
419     windows_total = len(samples)
420
421     # ---- cache setup / resume decision ----- #
422     if cache_dir is None:
423         cache_dir = default_cache_dir(out_csv)
424     os.makedirs(cache_dir, exist_ok=True)
425     det_jsonl, _ = _cache_paths(cache_dir)
426     cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
427     manifest = None if fresh else read_manifest(cache_dir)
428     resume = manifest_compatible(
429         manifest or {}, frames_dir=cache_key, weights=weights, sport=sport,
430         frames_in=frames_in, frames_total=len(frames),
431     )
432     if resume:
433         windows_done, det_by_fid = load_and_clean_cache(cache_dir)
434         logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
435     else:
436         if manifest is not None:
437             logger.info("cache incompatible with this run -> starting fresh")
438             reset_cache(cache_dir)
439             windows_done, det_by_fid = 0, defaultdict(list)
440
441     base_manifest = {
442         "version": CACHE_VERSION,
443         "frames_dir": cache_key,
444         "weights": weights,
445         "sport": sport,
446         "frames_in": frames_in,
447         "frames_total": len(frames),
448         "windows_total": windows_total,
449         "windows_done": windows_done,
450     }
451     write_manifest(cache_dir, base_manifest)
452
453     # ---- detector pass over the un-cached windows ----- #
454     remaining = samples[windows_done:]
455     if remaining:
456         _, transform_test = build_img_transforms(cfg)
457         try:
458             _, seq_transform_test = build_seq_transforms(cfg)
459         except Exception:
460             seq_transform_test = None
461         ds = ImageDataset(cfg, remaining, input_wh, output_wh,
462             transform=transform_test, seq_transform=seq_transform_test,
is_train=False)
463         # Streaming forces single-process loading: the in-memory frame store + the
464         # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.
465         loader_workers = 0 if streaming else num_workers
466         loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=loader_workers)
467         detector = build_detector(cfg)

```

```

468
469         from contextlib import ExitStack
470
471         t0 = time.time()
472         done = windows_done
473         win_idx = windows_done
474         log_every = max(1, log_every_batches)
475         flush_n = max(1, flush_every)
476         pending = []
477         logger.info(f"detecting on {len(remaining)} remaining windows "
478                   f"(total {windows_total}, batch={batch_size},
workers={loader_workers}, "
479                   f"source={'video-stream' if streaming else 'frames-dir'})...")
480         det_f = open(det_jsonl, "a")
481         try:
482             with ExitStack() as stack:
483                 stack.enter_context(torch.no_grad())
484                 # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
to the
485                 # in-memory decoded frame for that synthetic path; every other line of
486                 # __getitem__ runs unchanged so the tensor matches the PNG round-trip
exactly.
487                 # The detector's first needed frame is `windows_done` (window i starts
at
488                 # frame i), so prime the store there for a resume.
489                 if streaming:
490                     stack.enter_context(
491                         _streaming_read_image_patch(frame_loader, len(frames),
492                                                     window=frames_in,
start_index=windows_done))
493                 for bi, batch in enumerate(loader):
494                     imgs, _hms, trans = batch[0], batch[1], batch[2]
495                     img_paths = [list(t) for t in batch[-1]] # [frame_in][batch] ->
path
496                     batch_results, _ = detector.run_tensor(imgs, trans)
497                     nb = imgs.shape[0]
498                     for ib in range(nb):
499                         frames_payload = []
500                         for ie in sorted(batch_results[ib].keys()):
501                             p = img_paths[ie][ib]
502                             fid = _frame_id(p)
503                             plain = [_det_plain(d) for d in batch_results[ib][ie]]
504                             frames_payload.append([fid, plain])
505                             det_by_fid[fid].extend(plain)
506                             pending.append({"w": win_idx, "f": frames_payload})
507                             win_idx += 1
508                     done += nb
509                     if len(pending) >= flush_n or done >= windows_total:
510                         _append_windows(det_f, pending)
511                         pending = []
512                         base_manifest["windows_done"] = done
513                         write_manifest(cache_dir, base_manifest)
514                     if (bi + 1) % log_every == 0 or done >= windows_total:
515                         el = time.time() - t0
516                         new_done = done - windows_done
517                         rate = new_done / el if el > 0 else 0.0
518                         eta = (windows_total - done) / rate if rate > 0 else 0.0
519                         logger.info(f"{done}/{windows_total} "
520                                   f"({100 * done // max(1, windows_total)}%) |
{rate:.1f} win/s | "
521                                   f"elapsed {el:.0f}s | ETA {eta:.0f}s")
522                 if pending:
523                     _append_windows(det_f, pending)
524                     base_manifest["windows_done"] = done
525                     write_manifest(cache_dir, base_manifest)

```

```

526         finally:
527             det_f.close()
528             logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
529     else:
530         logger.info("detector cache already complete -> tracker-only re-run")
531
532     # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
533     tracker = build_tracker(cfg)
534     tracker.refresh()
535     rows = []
536     for fid in sorted(det_by_fid.keys()):
537         r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
538         rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
539
540     _atomic_write_trajectory(out_csv, rows)
541     visible = sum(1 for _, v, *_ in rows if v)
542     logger.info(f"wrote {len(rows)} rows -> {out_csv}")
543     logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
544     return out_csv
545
546
547 def extract_frames(video_path: str, frames_dir: str) -> str:
548     """Decode a video to PNG frames named by source frame index (00000000.png ...).
549
550     Idempotent: if the dir already holds the expected number of frames (matching the
551     container's reported frame count) we skip re-decoding ? re-extraction after a
552     reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
553     extracting with ffmpeg on the host and passing --frames_dir.
554     """
555     import cv2
556     os.makedirs(frames_dir, exist_ok=True)
557     cap = cv2.VideoCapture(video_path)
558     if not cap.isOpened():
559         raise SystemExit(f"cannot open video: {video_path}")
560     expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
561     existing = len(glob.glob(osp.join(frames_dir, "*.png")))
562     if expected > 0 and existing >= expected:
563         cap.release()
564         logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
565         return frames_dir
566     i = 0
567     while True:
568         ok, frame = cap.read()
569         if not ok:
570             break
571         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
572         i += 1
573     cap.release()
574     logger.info(f"extracted {i} frames -> {frames_dir}")
575     return frames_dir
576
577
578 def _probe_frame_count(video_path: str) -> int:
579     """Container-reported frame count via cv2 (0 if unknown). Used to size the
580     stream."""
581     import cv2
582     cap = cv2.VideoCapture(video_path)
583     if not cap.isOpened():
584         raise SystemExit(f"cannot open video: {video_path}")
585     n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
586     cap.release()
587     return n
588
589 def make_video_frame_loader(video_path: str):

```

```

590     """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.
591
592     ``frame_loader(index) -> BGR uint8 ndarray`` decodes the video forward-only with one
593     long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
594     detector's sliding-window order); it reads sequentially and only uses a frame-seek
595     to
596     skip forward when a *resume* starts past the current position. Each ``cap.read()``
597     returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
would
598     yield (PNG is lossless), so the detector sees identical pixels to the disk path.
599
600     Returns the count hint from the container so the caller can build the frame list;
601     ``release`` closes the capture.
602     """
603     import cv2
604     cap = cv2.VideoCapture(video_path)
605     if not cap.isOpened():
606         raise SystemExit(f"cannot open video: {video_path}")
607     count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
608     state = {"pos": 0} # next index cap.read() will return
609
610     def frame_loader(index: int):
611         index = int(index)
612         if index < state["pos"]:
613             raise RuntimeError(
614                 f"streaming decode is forward-only; got index {index} after
{state['pos']}")
615         if index > state["pos"]:
616             # Forward skip only happens when resuming past cached windows. Seek once.
617             cap.set(cv2.CAP_PROP_POS_FRAMES, index)
618             state["pos"] = index
619             ok, frame = cap.read()
620             if not ok:
621                 raise RuntimeError(f"failed to decode frame {index} from {video_path}")
622             state["pos"] += 1
623             return frame
624
625     def release():
626         cap.release()
627
628     return frame_loader, count_hint, release
629
630     def run_video_streaming(video_path: str, weights: str, out_csv: str, sport: str =
"badminton",
631                             limit: int = 0, batch_size: int = 8,
632                             log_every_batches: int = 50, cache_dir: "str | None" = None,
633                             flush_every: int = 1000, fresh: bool = False) -> str:
634         """Fast path: decode ``video_path`` on the fly and feed frames straight to the
635         detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
636         dominates a long clip). Output is bit-identical to the frames-on-disk path; see
637         ``run``'s docstring for why.
638         """
639         frame_loader, count_hint, release = make_video_frame_loader(video_path)
640         try:
641             if count_hint <= 0:
642                 raise SystemExit(
643                     f"could not determine frame count for {video_path}; cannot stream "
644                     f"(fall back to frame extraction with --frames_dir)")
645             frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
646             source_key = "video:" + osp.abspath(video_path)
647             return run(frame_list, weights, out_csv, sport, limit,
648                         batch_size=batch_size, num_workers=0,
649                         log_every_batches=log_every_batches, cache_dir=cache_dir,
650                         flush_every=flush_every, fresh=fresh,

```

```

651             frame_loader=frame_loader, source_key=source_key)
652         finally:
653             release()
654
655
656     def main():
657         ap = argparse.ArgumentParser()
658         ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
659         ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")
660         ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
661         ap.add_argument("--stream-video", action="store_true",
662             help="FAST PATH: decode --video on the fly and feed frames straight
to "
663                 "the detector (no PNG extraction). Output-equivalent to the
disk path.")
664         ap.add_argument("--weights", required=True)
665         ap.add_argument("--out", required=True)
666         ap.add_argument("--sport", default="badminton")
667         ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")
668         ap.add_argument("--batch-size", type=int, default=8)
669         ap.add_argument("--num-workers", type=int, default=4)
670         ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
671         ap.add_argument("--cache-dir", default=None,
672             help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
673         ap.add_argument("--flush-every", type=int, default=1000,
674             help="fsync the detector cache every N windows (bounds reboot
loss)")
675         ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
676         args = ap.parse_args()
677
678         if args.stream_video:
679             if not args.video:
680                 raise SystemExit("--stream-video requires --video")
681             run_video_streaming(args.video, args.weights, args.out, args.sport, args.limit,
682                               batch_size=args.batch_size,
683                               log_every_batches=args.log_every_batches,
cache_dir=args.cache_dir,
684                               flush_every=args.flush_every, fresh=args.fresh)
685             return
686
687         frames_dir = args.frames_dir
688         if args.video:
689             frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
690         if not frames_dir:
691             raise SystemExit("provide --frames_dir or --video")
692         run(frames_dir, args.weights, args.out, args.sport, args.limit,
693             batch_size=args.batch_size, num_workers=args.num_workers,
694             log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
695             flush_every=args.flush_every, fresh=args.fresh)
696
697
698     if __name__ == "__main__":
699         main()
700

```

8. assistant (2026-06-19T16:57:19.441Z)

This confirms claim (a) strongly. The tracker is imported `from trackers import build\_tracker`

(line 388) and the detector `from detectors import build\_detector` (line 386) ? both from the WASB-SBDT repo's packages, runs inside WSL. Let me look at how the tracker is built/used and check the det_raw.jsonl / trajectory CSV consumption, plus the licensing claims.

9. user (2026-06-19T16:57:21.198Z)

```
backend\pipeline\detectors\wasb_infer.py:386: from detectors import build_detector
backend\pipeline\detectors\wasb_infer.py:387: from trackers import build_tracker
backend\pipeline\detectors\wasb_infer.py:467:     detector = build_detector(cfg)
backend\pipeline\detectors\wasb_infer.py:533: tracker = build_tracker(cfg)
```

10. user (2026-06-19T16:57:21.838Z)

Found 30 files

```
backend\eval\rally_seg_eval.py
backend\eval\golden_real_fixtures.py
backend\eval\golden_fixtures.py
backend\config\models.py
backend\pipeline\detectors\tracknet_runner.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\eval>windowing.py
backend\pipeline\detectors\wasb_infer.py
backend\pipeline\detectors\wasb_runner.py
backend\pipeline\detectors\base.py
backend\eval\serve_contrast.py
backend\eval\distill_local.py
backend\eval\served_gate_a.py
backend\eval\fusion_golden.py
backend\pipeline\detectors\person_detector.py
backend\pipeline\detectors\fusion_features.py
backend\pipeline\detectors\rally_gate.py
backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\eval\calibrate_local.py
backend\pipeline\detectors\__init__.py
backend\features\rally_seq.py
backend\eval\rally_seq_proto.py
backend\eval\eval_partial_labels.py
backend\eval\calibration.py
backend\eval\calibrate_wasb.py
backend\eval\export_wasb_segments.py
backend\eval\audio\e1_probe.py
backend\eval\gemini_refine.py
```

11. user (2026-06-19T16:57:22.574Z)

Found 52 files

```
docs\PROMINENT_COURT_DETECTION.md
docs\README.md
docs\RALLY_QUALITY_RESEARCH.md
docs\RALLY_DETECTION_QUALITY_REPORT.md
docs\RALLY_DETECTION_GAP_CLOSURE.md
docs\QUALITY_ITERATIONS.md
docs\NEXT_STEPS.md
docs\MULTI_SIGNAL_FUSION_PLAN.md
docs\GOLDEN_VIDEOS.md
docs\HOW_RALLY_DETECTION_WORKS.md
docs\CODE_MAP.md
docs\BACKLOG.md
docs\DESKTOP_APP_PLAN.md
docs\archives\quality-iterations\README.md
docs\archives\quality-iterations\2026-06-18-lovo-resweep-n13\README.md
docs\VIDEO_RECORDING_GUIDELINES.md
```

docs\REAL_FOOTAGE_VALIDATION_REPORT.md
docs\GOLDEN_REGRESSION_FIXTURES.md
docs\GOLDEN_DATA_SHARING.md
docs\POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md
docs\ROADMAP.md
docs\TRACKNET_WSL_SETUP.md
docs\PLATFORM_ARCHITECTURE.md
docs\ANALYZERS_AND_RECONCILERS.md
docs\COMMERCIALIZATION.md
docs\EVALUATION.md
docs\EXECUTION_POLICY.md
docs\EXPERIMENT_HARNESS.md
docs\archives\quality-iterations\2026-06-first-ab-experiment\README.md
docs\STORAGE_SHARING_MODEL.md
docs\GOLDEN_SET_VENDORING.md
docs\VIDEO_LOCALITY_MODEL.md
docs\PMF_FIELD_SIGNALS.md
docs\I18N_PLAN.md
docs\UI_ALPHA_EXECUTION_PLAN.md
docs\archives\decisions\DECISIONS.md
docs\UI_PROPOSAL.md
docs\HOSTING_PLAN.md
docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md
docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md
docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md
docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md
docs\archives\quality-iterations\2026-06-initial\README.md
docs\archives\checkpoints\2026-06-strategy-foundation\README.md
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md
docs\archives\research\AUDIO_E1_FINDINGS.md
docs\archives\research\AUDIO_RALLY_DETECTION_PLAN.md
docs\archives\research\RALLY_DETECTION_RESEARCH.md
docs\archives\COMMERCIAL_READINESS_REVIEW.md
docs\archives\MONETIZATION_AUDIT.md
docs\GOLDEN_SET_PHASE1_VIDEOS.md
docs\GOLDEN_SET_IMPLEMENTATION.md

12. assistant (2026-06-19T16:57:26.202Z)

`build\_tracker` and `build\_detector` are imported only inside `wasb\_infer.py` (which runs in WSL). Let me verify how the trajectory CSV is consumed (the pure-Python path), how `wasb\_runner.py` stages into WSL, and read the licensing docs. Let me also check the parse_trajectory_csv consumer.

13. user (2026-06-19T16:57:27.583Z)

```
1      """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3      This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4      inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6          1. Translate the Windows video path to its WSL-visible form (and optionally
7             stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8          2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9          3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10         4. Convert that trajectory into high-motion "action windows" ? the same
11            candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12            pipeline (AI handoff, DB population) is unchanged.
13
14      Nothing here imports TensorFlow; all model code stays in WSL.
15      """
16
17      import os
18      import csv
19      import shlex
```

```

20     import logging
21     import subprocess
22     from dataclasses import dataclass
23     from typing import List, Dict, Any, Optional, Tuple
24
25     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27     logger = logging.getLogger(__name__)
28
29
30     @dataclass
31     class TrackNetConfig:
32         """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "TrackNetV4"
37         weights_path: str = ""                        # WSL path to the .h5/.keras weights
(REQUIRED)
38         queue_length: int = 5
39         stage_in_wsl: bool = True                    # copy video into WSL fs first
(perf)
40         wsl_stage_dir: str = "~/clips"                # where staged videos/outputs live
in WSL
41         timeout_sec: int = 1800                      # 30 min; kills a hung call (0 = no
timeout)
42         keep_frames: bool = False                    # retain staged video after success
(debug only)
43         min_free_gb: float = 0.0                     # warn if WSL free space below this
(0 = off)
44
45     @classmethod
46     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47         tn = (idx_cfg or {}).get("tracknet", {}) or {}
48         return cls(
49             distro=tn.get("wsl_distro", "Ubuntu"),
50             repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51             conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52             conda_env=tn.get("conda_env", "TrackNetV4"),
53             weights_path=tn.get("weights_path", ""),
54             queue_length=int(tn.get("queue_length", 5)),
55             stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56             wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57             timeout_sec=int(tn.get("timeout_sec", 1800)),
58             keep_frames=bool(tn.get("keep_frames", False)),
59             min_free_gb=float(tn.get("min_free_gb", 0.0)),
60         )
61
62
63     def to_wsl_mnt_path(win_path: str) -> str:
64         """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66         Already-POSIX paths (starting with / or ~) are returned unchanged so the
67         same helper is safe to call on either kind of path.
68         """
69         p = str(win_path)
70         if p.startswith("/") or p.startswith("~"):
71             return p
72         p = p.replace("\\", "/")
73         if len(p) >= 2 and p[1] == ":":
74             drive = p[0].lower()
75             return f"/mnt/{drive}{p[2:]}"
76         return p

```

```

77
78
79 @dataclass
80 class TrajectoryPoint:
81     frame: int
82     visible: bool
83     x: float
84     y: float
85
86
87 class TrackNetRunner(WslCommandMixin, DetectorRunner):
88     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90     Implements the common DetectorRunner contract so a future Linux-native or
91     alternative trajectory runner can be substituted without touching the hybrids.
92     """
93
94     def __init__(self, cfg: TrackNetConfig):
95         self.cfg = cfg
96
97     def healthcheck(self) -> Tuple[bool, str]:
98         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100         Returns (ok, message). Cheap to call before a long run.
101         """
102         cmd = (
103             f"conda activate {self.cfg.conda_env} && "
104             f"python -c \"import tensorflow as tf; "
105             f"print('TF', tf.__version__, 'GPUs', "
106             f"len(tf.config.list_physical_devices('GPU')))\\"
107         )
108         try:
109             res = self._wsl_bash(cmd)
110         except FileNotFoundError:
111             return False, "`wsl` not found ? is this running on Windows with WSL2
112             installed?"
113         except subprocess.TimeoutExpired:
114             return False, "WSL healthcheck timed out."
115         out = (res.stdout or "").strip()
116         if res.returncode != 0:
117             return False, f"WSL env check failed: {(res.stderr or out).strip()}"
118         return True, out
119
120 # ----- inference
121
122 def run_predict(self, video_win_path: str, output_win_dir: str,
123                 video_id: Optional[str] = None) -> Optional[str]:
124     """Run predict.py on a video. Returns the Windows path to the produced CSV, or
125     None.
126
127     ``output_win_dir`` is a Windows directory (under the repo's output/) that
128     WSL writes into via its /mnt view, so the Windows process can read the CSV
129     back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
130     therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
131     a filename cannot collide on the output CSV.
132     """
133     if not self.cfg.weights_path:
134         logger.error(
135             "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
136             "in config.json to the WSL path of your trained TrackNetV4 weights."
137         )
138     return None

```

```

139     # predict.py cwd instead of the repo (same bug class as wasb_runner).
140     output_win_dir = os.path.abspath(output_win_dir)
141     os.makedirs(output_win_dir, exist_ok=True)
142     # Normalize separators so basename/splitext work when tests (or collector)
143     # pass Windows-style paths on a Linux host.
144     video_win_path = str(video_win_path).replace("\\", "/")
145     base = os.path.basename(video_win_path)
146     stem, ext = os.path.splitext(base)
147
148     # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
149     if ext.lower() not in (".mp4", ".avi"):
150         logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
151         return None
152
153     wsl_out = to_wsl_mnt_path(output_win_dir)
154     # Namespace by video_id so two same-filename videos don't collide on the CSV.
155     # predict.py derives its output name from the (staged) video stem, so staging
156     # under the namespaced name propagates into "<key>_predict.csv".
157     key = f"{stem}_{video_id[:12]}" if video_id else stem
158     expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
159
160     # Resolve the video path WSL will read.
161     if self.cfg.stage_in_wsl:
162         staged = self._stage_video(video_win_path, f"{key}{ext}")
163         if staged is None:
164             return None
165         wsl_video = staged
166     else:
167         # No staging: predict.py reads /mnt directly and writes
168         "<stem>_predict.csv".
169         # We cannot rename its output, so fall back to the un-namespaced name.
170         wsl_video = to_wsl_mnt_path(video_win_path)
171         expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
172
173     if self.cfg.min_free_gb > 0:
174         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
175         if free is not None and free < self.cfg.min_free_gb:
176             logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
177                             "consider running tools/cleanup_caches.py.",
178                             free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
179
180     cmd = (
181         f"conda activate {self.cfg.conda_env} && "
182         f"cd {self.cfg.repo_dir}/src && "
183         f"python predict.py "
184         f"--video_path {wsl_tilde_quote(wsl_video)} "
185         f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
186         f"--output_dir {wsl_tilde_quote(wsl_out)} "
187         f"--queue_length {self.cfg.queue_length}"
188     )
189     logger.info("Running TrackNet inference on %s ...", base)
190     try:
191         res = self._wsl_bash(cmd)
192     except subprocess.TimeoutExpired:
193         logger.error("TrackNet inference timed out after %ss.",
194                     self.cfg.timeout_sec)
195         return None
196
197     if res.returncode != 0:
198         logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
199                     res.stdout)[-2000:])
200         return None
201
202     if not os.path.exists(expected_csv_win):
203         logger.error("TrackNet finished but expected CSV not found at %s",

```

```

expected_csv_win)
201         return None
202         logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
203
204         # Success ? the CSV is the durable output; drop the staged video copy (a pure,
205         # regenerable intermediate). On earlier failure we return above without
cleaning.
206         if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
207             logger.info("Cache hygiene: removing staged video for %s "
208                         "(set indexing.tracknet.keep_frames=true to retain).", key)
209             self._wsl_rm_rf([wsl_video])
210         return expected_csv_win
211
212     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
213         """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
214
215         Returns the WSL path to the staged copy, or None on failure.
216         """
217         src_mnt = to_wsl_mnt_path(video_win_path)
218         reason = self.wsl_source_unreadable_reason(src_mnt)
219         if reason:
220             logger.error("Cannot stage video into WSL: %s", reason)
221             return None
222         dest = f"{self.cfg.wsl_stage_dir}/{base}"
223         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}"
224         {wsl_tilde_quote(dest)} && echo OK"
225         try:
226             res = self._wsl_bash(cmd)
227         except subprocess.TimeoutExpired:
228             logger.error("Staging video into WSL timed out.")
229             return None
230         if res.returncode != 0 or "OK" not in (res.stdout or ""):
231             logger.error("Failed to stage video into WSL: %s", (res.stderr or
232             res.stdout).strip())
233             return None
234         return dest
235
236     # ----- CSV -> windows
237
238     @staticmethod
239     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
240         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
241         points: List[TrajectoryPoint] = []
242         with open(csv_win_path, newline="") as f:
243             reader = csv.DictReader(f)
244             for row in reader:
245                 try:
246                     points.append(TrajectoryPoint(
247                         frame=int(row["Frame"]),
248                         visible=int(row["Visibility"]) == 1,
249                         x=float(row["X"]),
250                         y=float(row["Y"]),
251                     ))
252                 except (KeyError, ValueError):
253                     continue
254             points.sort(key=lambda p: p.frame)
255         return points
256
257     @staticmethod
258     def _active_frames(points: List[TrajectoryPoint], fps: float, frame_width: float,
259                       velocity_thresh: float) -> List[Tuple[int, float]]:
260         """Per-frame ``(frame_idx, velocity)`` for frames where the shuttle is in
flight.
261
262         Computes each visible point's normalised velocity from the last *visible* point;

```

```

261         an absent shuttle breaks the trajectory. A frame qualifies when its velocity
exceeds
262         ``velocity_thresh``. ``fps``/``frame_width`` are expected to be already
defaulted by
263         the caller so identical floats feed the velocity math."""
264         active = [] # (frame_idx, velocity)
265         prev: Optional[TrajectoryPoint] = None
266         for p in points:
267             if not p.visible:
268                 prev = None # break the trajectory; absent shuttle = not in flight
269                 continue
270             if prev is not None:
271                 dframes = p.frame - prev.frame
272                 if dframes > 0:
273                     dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
274                     velocity = (dist / frame_width) * (fps / dframes)
275                     if velocity > velocity_thresh:
276                         active.append((p.frame, velocity))
277             prev = p
278         return active
279
280     @staticmethod
281     def trajectory_to_action_windows(
282         points: List[TrajectoryPoint],
283         fps: float,
284         frame_width: float,
285         chunk_start: float = 0.0,
286         velocity_thresh: float = 0.02,
287         merge_gap: float = 2.0,
288         min_window_duration: float = 2.0,
289         chunk_index: Optional[int] = None,
290         max_window_duration: float = 0.0,
291         min_split_gap: float = 0.5,
292         window_hard_cap: bool = False,
293         window_inactivity_end: float = 0.0,
294         inactivity_min_density: float = 0.34,
295         inactivity_rally_thresh: float = 0.08,
296         window_blob_fallback: bool = False,
297         window_blob_factor: float = 4.0,
298     ) -> List[Dict[str, Any]]:
299         """Convert a shuttle trajectory into candidate rally windows.
300
301         A frame is "active" when the shuttle is visible AND its inter-frame
302         displacement (normalised by frame width, per second) exceeds
303         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
304         ``merge_gap`` seconds of each other are grouped into one window; short
305         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
306         shape feeding the AI-handoff phase is identical.
307
308         ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
against
309         the *over-merge* failure where "shuttle moving" is conflated with "rally in
progress"
310         and a single window swallows several rallies + the dead time between them. When
311         window longer than this is recursively SPLIT at its largest internal inactivity
gap (the
312         longest stretch with no in-flight shuttle ? the most likely rally boundary),
provided
313         that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
fix from
314         ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
recall cannot
315         drop, and it is config-gated default-OFF until measured on the golden set.
316

```

```

317         ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
an
318         over-long window has NO internal inactivity gap to split on (a continuously-
active
319         trajectory ? the shuttle never stops moving, e.g. the real-footage
``mahadevpura-1``
320         blob where the gap-splitter alone left a single 174 s window), it is recursively
321         hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
cuts
322         (never merges), so recall cannot drop; gated default-OFF until measured.
323
324         ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
unlike
325         ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
every gap-less
326         over-cap window and over-segments normal clips), this runs AFTER the R4
inactivity trim and
327         force-chops ONLY a genuine *blob* ? a group still longer than
``window_blob_factor`` *
328         ``max_window_duration`` once the gap-split and the principled rally-end trim
have both had
329         their chance (e.g. mahadevpura-1's ~65 s residual ? 11× a 6 s cap; a 7 s rally
is left
330         alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
logged, and
331         each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
332         rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
333         can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
334         ""
335         if fps <= 0:
336             fps = 30.0
337         if frame_width <= 0:
338             frame_width = 1280.0
339
340         active = TrackNetRunner._active_frames(points, fps, frame_width,
velocity_thresh)
341
342         if not active:
343             return []
344
345         max_gap_frames = int(merge_gap * fps)
346
347         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
348             vels = [v for _, v in group]
349             return {
350                 "start": group[0][0] / fps + chunk_start,
351                 "end": group[-1][0] / fps + chunk_start,
352                 "active_frame_count": len(group),
353                 "peak_velocity": max(vels),
354                 "mean_velocity": sum(vels) / len(vels),
355                 "track_id_count": 1, # single shuttle; kept for window-shape parity
356                 "chunk_index": chunk_index,
357             }
358
359         groups: List[List[Tuple[int, float]]] = []
360         group = [active[0]]
361         for af in active[1:]:
362             if af[0] - group[-1][0] <= max_gap_frames:
363                 group.append(af)
364             else:
365                 groups.append(group)
366                 group = [af]

```

```

367         groups.append(group)
368
369         if max_window_duration > 0:
370             groups = TrackNetRunner._split_overlong_groups(
371                 groups, int(max_window_duration * fps), int(min_split_gap * fps),
372                 hard_cap=window_hard_cap)
373
374         # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
375         # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
376         # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
377         # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
378         # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
379         if window_inactivity_end > 0:
380             tw = int(window_inactivity_end * fps)
381             groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
382                                                         inactivity_rally_thresh) for g
in groups]
383
384         # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
385         # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
386         # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
387         # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
388         forced_flags = [False] * len(groups)
389         if window_blob_fallback and max_window_duration > 0:
390             groups, forced_flags = TrackNetRunner._blob_fallback_chop(
391                 groups, int(max_window_duration * fps), window_blob_factor)
392
393         windows = []
394         for g, forced in zip(groups, forced_flags):
395             w = _finalize(g)
396             if forced:
397                 w["forced_cut"] = True
398             windows.append(w)
399         return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
400
401     @staticmethod
402     def _trim_inactive_tail(group: List[Tuple[int, float]], trail_frames: int,
403                             min_density: float, rally_thresh: float) -> List[Tuple[int,
float]]:
404         """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
405         (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
windowing
406         over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
``rally_thresh``
407         (higher than the active threshold). The rally is "on" at frame ``i`` while its
trailing
408         ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
back to the
409         LAST such frame ? everything after is sustained slow drift (rally over). Only
cuts (recall
410         can't rise); a window whose tail never goes slow is untouched, and if no fast-
dense point
411         exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
pointer; the START
412         is left alone (already ~Gemini-accurate per the n=2 boundary-error finding)."""

```

```

413         if trail_frames <= 0 or len(group) < 2:
414             return group
415         frames = [f for f, _ in group]
416         fast = [1 if v > rally_thresh else 0 for _, v in group]
417         lo = 0
418         fast_sum = 0
419         last_dense = -1
420         for i in range(len(frames)):
421             fast_sum += fast[i]
422             while frames[lo] <= frames[i] - trail_frames:
423                 fast_sum -= fast[lo]
424                 lo += 1
425             win_count = i - lo + 1
426             if win_count > 0 and fast_sum / win_count >= min_density:
427                 last_dense = i
428         return group[: last_dense + 1] if last_dense >= 0 else group
429
430     @staticmethod
431     def _midpoint_split_index(group: List[Tuple[int, float]]) -> int:
432         """Index of the active frame nearest the group's temporal midpoint (used to chop
433         a
434         gap-less group into two). Shared by the W1 hard-cap fallback and the R5 blob-
435         fallback."""
436         mid = (group[0][0] + group[-1][0]) / 2.0
437         split_i = min(range(1, len(group)), key=lambda i: abs(group[i][0] - mid))
438         return split_i
439
440     @staticmethod
441     def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
442         int,
443         min_split_frames: int,
444         hard_cap: bool = False) -> List[List[Tuple[int, float]]]:
445         """Recursively split any active-frame group longer than ``max_win_frames`` at
446         its largest
447         internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
448         likely rally
449         boundary. Splitting only subdivides existing groups (never merges/extends), so
450         it cannot
451         invent or drop coverage; a group with no gap ? the floor is left intact (a
452         genuinely long
453         rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
454         its
455         temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
456         true upper
457         bound on continuously-active trajectories). Iterative worklist to bound stack
458         depth."""
459         if max_win_frames <= 0:
460             return groups
461         out: List[List[Tuple[int, float]]] = []
462         work = list(groups)
463         while work:
464             g = work.pop()
465             if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
466                 out.append(g)
467                 continue
468             best_i, best_gap = -1, -1
469             for i in range(1, len(g)):
470                 gap = g[i][0] - g[i - 1][0]
471                 if gap > best_gap:
472                     best_gap, best_i = gap, i
473             if best_i < 0 or best_gap < min_split_frames:
474                 # Too long but no real dead-time gap. Default: keep as one (a genuine
475                 long rally).
476                 out.append(g)
477             else:
478                 # hard_cap: chop at the temporal midpoint so the cap is actually
479                 enforced.

```

```

466         if hard_cap:
467             split_i = TrackNetRunner._midpoint_split_index(g)
468             work.append(g[:split_i])
469             work.append(g[split_i:])
470         else:
471             out.append(g)
472             continue
473         work.append(g[:best_i])
474         work.append(g[best_i:])
475     out.sort(key=lambda grp: grp[0][0])
476     return out
477
478     @staticmethod
479     def _blob_fallback_chop(groups: List[List[Tuple[int, float]]], max_win_frames: int,
480                             blob_factor: float = 4.0
481                             ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
482         """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
483         inactivity trim have each had their chance, a group still longer than
``blob_factor`` x
484         ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
485         signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
486         11x a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
(~1?2x the cap)
487         is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
488         midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
489         degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
490         rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
491         can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
492         that piece came from a forced chop. Iterative worklist to bound stack depth."""
493         if max_win_frames <= 0:
494             return groups, [False] * len(groups)
495         blob_frames = max_win_frames * max(1.0, blob_factor)
496         out: List[List[Tuple[int, float]]] = []
497         flags: List[bool] = []
498         forced = 0
499         for g in groups:
500             if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
501                 out.append(g)
502                 flags.append(False)
503                 continue
504             pieces: List[List[Tuple[int, float]]] = []
505             work = [g]
506             while work:
507                 h = work.pop()
508                 if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
509                     pieces.append(h)
510                     continue
511                 split_i = TrackNetRunner._midpoint_split_index(h)
512                 work.append(h[:split_i])
513                 work.append(h[split_i:])
514             pieces.sort(key=lambda grp: grp[0][0])
515             out.extend(pieces)
516             flags.extend([True] * len(pieces))
517             forced += len(pieces) - 1
518         if forced:
519             logger.warning(
520                 "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "

```

```

521             "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /
two-sided "
522             "motion), not a bigger cap", forced)
523     return out, flags
524

```

14. user (2026-06-19T16:57:28.028Z)

```

1     """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4         1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5         2. copies wasb_infer.py into the WASB repo's src/ dir,
6         3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7         4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9     So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10    with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11    """
12    import os
13    import shlex
14    import logging
15    import subprocess
16    from dataclasses import dataclass
17    from typing import Dict, Any, Optional, Tuple
18
19    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20    # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21    from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22    from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
23
24    logger = logging.getLogger(__name__)
25
26    # Windows path to the inference wrapper we stage into WSL.
27    _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30    @dataclass
31    class WasbConfig:
32        """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33        distro: str = "Ubuntu"
34        repo_dir: str = "~/models/WASB-SBDT"
35        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36        conda_env: str = "wasb"
37        weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38        sport: str = "badminton"
39        wsl_stage_dir: str = "~/clips"
40        timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41        keep_frames: bool = False # retain staged video + frame cache after success (debug
only)
42        min_free_gb: float = 0.0 # warn if WSL free space below this before a run (0 =
off)
43
44        # FAST PATH (default OFF until verified): decode the staged video on the fly inside
45        # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
46        # extraction that dominates a long clip. Output is bit-identical to the PNG path
47        # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
it.
48
49        stream_video: bool = False
50
51    @classmethod
52    def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":

```

```

51         w = (idx_cfg or {}).get("wasb", {}) or {}
52         return cls(
53             distro=w.get("wsl_distro", "Ubuntu"),
54             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
55             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
56             conda_env=w.get("conda_env", "wasb"),
57             weights_path=w.get("weights_path",
58                               "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
59             sport=w.get("sport", "badminton"),
60             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
61             timeout_sec=int(w.get("timeout_sec", 1800)),
62             keep_frames=bool(w.get("keep_frames", False)),
63             min_free_gb=float(w.get("min_free_gb", 0.0)),
64             stream_video=bool(w.get("stream_video", False)),
65         )
66
67
68     class WasbRunner(WslCommandMixin, DetectorRunner):
69         def __init__(self, cfg: WasbConfig):
70             self.cfg = cfg
71
72         def healthcheck(self) -> Tuple[bool, str]:
73             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
74             cmd = (f"conda activate {self.cfg.conda_env} && "
75                  f"python -c \"import torch; print('torch', torch.__version__, "
76                  f"'cuda', torch.cuda.is_available())\"")
77             try:
78                 res = self._wsl_bash(cmd)
79             except FileNotFoundError:
80                 return False, "`wsl` not found ? Windows + WSL2 required."
81             except subprocess.TimeoutExpired:
82                 return False, "WSL healthcheck timed out."
83             out = (res.stdout or "").strip()
84             if res.returncode != 0:
85                 return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
86             return True, out
87
88         def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
89             src_mnt = to_wsl_mnt_path(video_win_path)
90             reason = self.wsl_source_unreadable_reason(src_mnt)
91             if reason:
92                 logger.error("Cannot stage video into WSL: %s", reason)
93                 return None
94             dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95             cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
96             try:
97                 res = self._wsl_bash(cmd)
98             except subprocess.TimeoutExpired:
99                 logger.error("Staging video into WSL timed out.")
100                 return None
101             if res.returncode != 0 or "OK" not in (res.stdout or ""):
102                 logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
103                 return None
104             return dest
105
106         def _stage_infer_script(self) -> bool:
107             """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
108             src_mnt = to_wsl_mnt_path(_INFER_WIN)
109             cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
OK"
110             res = self._wsl_bash(cmd)
111             return res.returncode == 0 and "OK" in (res.stdout or "")

```

```

112
113 def run_predict(self, video_win_path: str, output_win_dir: str,
114                 video_id: Optional[str] = None) -> Optional[str]:
115     """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
116
117     All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
118     two videos that share a filename (e.g. two ``match.mp4`` from different folders)
119     can never reuse each other's staged frames, resumable cache, or CSV.
120     """
121     # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
122     # translation: to_wsl_mnt_path passes relative paths through unchanged, so
123     # WSL resolved them against the inference cwd (~/.models/WASB-SBDT/src) and
124     # the CSV landed inside WSL while Windows polled the repo-relative path.
125     output_win_dir = os.path.abspath(output_win_dir)
126     os.makedirs(output_win_dir, exist_ok=True)
127     # Normalize for cross-platform basename (tests use Windows paths on Linux).
128     video_win_path = str(video_win_path).replace("\\", "/")
129     base = os.path.basename(video_win_path)
130     stem, ext = os.path.splitext(base)
131     # Unique, content-derived key: same filename + different bytes -> different key.
132     key = f"{stem}__{video_id[:12]}" if video_id else stem
133     wsl_out_dir = to_wsl_mnt_path(output_win_dir)
134     expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
135
136     if not self._stage_infer_script():
137         logger.error("Could not stage wasb_infer.py into WSL.")
138         return None
139     staged_video = self._stage_video(video_win_path, f"{key}{ext}")
140     if staged_video is None:
141         return None
142     frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
143
144     # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
145     # WSL stage filesystem is already low so the user can clean up before it fills.
146     # (Streaming never materializes the PNGs, so the guard only matters off the fast
147     path.)
148     if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
149         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
150         if free is not None and free < self.cfg.min_free_gb:
151             logger.warning(
152                 "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
153                 extraction "
154                 "may fill the disk ? consider running tools/cleanup_caches.py.",
155                 free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
156
157     # Fast path: stream-decode the video in-process (no PNG extraction). Output is
158     # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
159     clip.
160     if self.cfg.stream_video:
161         frame_args = "--stream-video "
162     else:
163         frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
164     cmd = (
165         f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
166         f"python wasb_infer.py "
167         f"--video {wsl_tilde_quote(staged_video)} "
168         f"{frame_args}"
169         f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
170         f"--sport {shlex.quote(self.cfg.sport)} "
171         f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}")
172     )
173     logger.info("Running WASB inference on %s ...", base)
174     try:
175         res = self._wsl_bash(cmd)
176     except subprocess.TimeoutExpired:

```

```

174         logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
175         return None
176     if res.returncode != 0:
177         logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
178         return None
179     if not os.path.exists(expected_csv_win):
180         logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
181         return None
182     logger.info("WASB trajectory CSV: %s", expected_csv_win)
183
184     # Success ? the trajectory CSV is the durable output; the staged video copy and
185     # the (huge) decoded frame cache are now pure intermediates. Delete them unless
186     # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
187     # so the frame cache + detector cache remain for a resumable re-run.
188     # (Streaming never writes a frame cache, so there's only the staged video to
drop.)
189     if not self.cfg.keep_frames:
190         stale = [staged_video] if self.cfg.stream_video else [frames_dir,
staged_video]
191         logger.info("Cache hygiene: removing staged video%s for %s "
192                     "(set indexing.wasb.keep_frames=true to retain).",
193                     "" if self.cfg.stream_video else " + frame cache", key)
194         self._wsl_rm_rf(stale)
195         return expected_csv_win
196
197     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
198     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
199     trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
200

```

15. assistant (2026-06-19T16:57:33.833Z)

This is very clear now. The ``parse_trajectory_csv`` and ``trajectory_to_action_windows`` are pure-Python `@staticmethod`s` on ``TrackNetRunner`` (no torch/cv2/WSL imports in those functions ? they just use ``csv``, ``math``, etc.), inherited by ``WasbRunner``. They consume the ``Frame,Visibility,X,Y`` CSV that ``wasb_infer.py`` produces.

Now let me confirm the licensing claims. Let me search the docs for WASB-SBDT license assertions and check the WASB-SBDT repo's own LICENSE if it's checked out locally.

16. user (2026-06-19T16:57:35.813Z)

```

docs\COMMERCIALIZATION.md:42:[Omitted long matching line]
docs\COMMERCIALIZATION.md:65:| WASB-SBDT (shuttle) | MIT (code) | No | ? code clear |
Weights provenance open (C3). |
docs\DESKTOP_APP_PLAN.md:38:| D4 | Commercial-clean licensing for everything bundled: MIT /
Apache-2.0 / BSD only (code, weights). ffmpeg ships as an LGPL build (no GPL-only encoders
like x264). | `COMMERCIALIZATION` (ratified 2026-06-09); `VIDEO_LOCALITY_MODEL` OQ5 ("ffmpeg
LGPL build = licensing-clean") | WASB-SBDT weights are MIT `[verified]` (see `tracknet-wasb-
wsl2-decision`). The stitch must use openh264 (BSD) or OS hardware encoders, not statically-
linked x264. See §3.4. |
docs\DESKTOP_APP_PLAN.md:51:[Omitted long matching line]
docs\DESKTOP_APP_PLAN.md:65:- Weights: WASB-SBDT = MIT `[verified]` (`tracknet-wasb-
wsl2-decision`), redistributable.
docs\archives\MONETIZATION_AUDIT.md:38:> ? Correction to ADR-004's open question: WASB's
commercial-use license was the flagged unknown. WASB-SBDT (`nttcom/WASB-SBDT`) is MIT ? CLEAR
for commercial use.The shuttle path is licensing-safe; only MonoTrack is off-limits.
docs\archives\MONETIZATION_AUDIT.md:57:- WASB-SBDT (`nttcom`, MIT) or TrackNetV3
(`qaz812345` or `alenzex`, both MIT) ? all CLEAR for commercial code use.
docs\archives\MONETIZATION_AUDIT.md:124:- Shuttle: `nttcom/WASB-SBDT` (MIT),
`qaz812345/TrackNetV3` (MIT), `alenzex/TrackNetV3` (MIT), `jhwang7628/monotrack` (Adobe

```

```

Research, noncommercial).
docs\CODE_MAP.md:145:- `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name
`'wasb_hybrid'`, the LIVE shuttle substrate; MIT, ships pretrained badminton weights) ? thin
subclass: `family='wasb'`, wires `WasbRunner`/`WasbConfig`; `detection_params` extras =
`weights_path` only.
docs\CODE_MAP.md:158:[Omitted long matching line]
docs\CODE_MAP.md:264:## 4. WASB WSL CONTRACT (external dep `~/models/WASB-SBDT`, MIT; NOT in
this repo ? claims below are unverified against external source, only the repo-side caller
wasb_infer.py was checked)
docs\CODE_MAP.md:265:- Env: WSL conda `wasb`, cwd `~/models/WASB-SBDT/src`. Hydra
`compose(config_name='eval', overrides=[dataset=<sport>, model=wasb,
detector.model_path=<weights>, runner.device=cuda, runner.gpus=[0]])`. `gpus=[0]` because box is
single-GPU. (overrides VERIFIED in @backend/pipeline/detectors/wasb_infer.py::_build_cfg.)
docs\CODE_MAP.md:270:- Weights `pretrained_weights/wasb_badminton_best.pth.tar` ? NOT committed,
academic-data-trained -> ? confirm terms before COMMERCIAL deploy. monotrack weights =
noncommercial, avoided.
docs\archives\decisions\DECISIONS.md:28: https://github.com/nttcom/WASB-SBDT
docs\archives\decisions\DECISIONS.md:244:2. **Shuttle tracking stays permissive.** **WASB-SBDT
and TrackNetV3 are MIT (CLEAR for commercial use)** ? this resolves the licensing unknown
flagged in ADR-001/ADR-004. **MonoTrack (Adobe Research, noncommercial) and YOLO-NAS weights
(Deci, noncommercial) are banned.** Train our own checkpoints (or confirm per-checkpoint terms),
since distributed research weights/datasets aren't covered by the MIT *code* grants.
docs\archives\decisions\DECISIONS.md:359:frames ?? WASB (frozen, MIT) ?? shuttle trajectory ??
docs\MULTI_SIGNAL_FUSION_PLAN.md:87:frames ?? WASB (frozen, MIT) ?? shuttle trajectory ???
docs\MULTI_SIGNAL_FUSION_PLAN.md:236:| Shuttle: WASB-SBDT, TrackNetV3 | MIT | frozen badminton
weights | ? (ADR-001/002) |
docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md:122:- **?? WASB checkpoint provenance (C3)
propagates.** The study calls WASB "MIT," but that's the *code*; the
docs\archives\research\RALLY_DETECTION_RESEARCH.md:43: terms were NOT confirmed ? must verify
before commercial use. (WASB code = MIT; weights = academic-trained.)
docs\archives\research\RALLY_DETECTION_RESEARCH.md:78:2307.16000 · MLLM4WTAL arXiv:2411.08466 ·
action-spotting survey arXiv:2505.03991 · WASB-SBDT (github nttcom)
docs\RALLY_DETECTION_QUALITY_REPORT.md:16:shuttle-trajectory substrate** (a WASB CNN that emits
per-frame shuttle position), from which a
docs\RALLY_DETECTION_QUALITY_REPORT.md:86:**Layer 1 ? Detection ("where is the shuttle this
frame?").** A frozen WASB CNN emits a per-frame
docs\TRACKNET_WSL_SETUP.md:14:| WASB repo | `~/models/WASB-SBDT`
(https://github.com/nttcom/WASB-SBDT) |
docs\TRACKNET_WSL_SETUP.md:101:~/models/WASB-SBDT/pretrained_weights/`:

```

17. user (2026-06-19T16:57:36.514Z)

```

1      # TrackNet / WASB on WSL2 ? setup & usage
2
3      Self-hosted shuttle-trajectory detectors run inside WSL2 (Ubuntu) and are bridged
4      to the Windows indexer via a subprocess adapter. See
`docs/archives/decisions/DECISIONS.md` (ADR-001)
5      for the why.
6
7      ## What's installed (WSL2 / Ubuntu 24.04)
8
9      | Thing | Location (WSL) |
10     |-----|-----|
11     | Miniconda | `~/miniconda3` |
12     | TrackNetV4 repo | `~/models/TrackNetV4` (https://github.com/TrackNetV4/TrackNetV4) |
13     | TrackNetV4 conda env | `TrackNetV4` (Python 3.9, TensorFlow 2.17 + bundled CUDA 12.3)
|
14     | WASB repo | `~/models/WASB-SBDT` (https://github.com/nttcom/WASB-SBDT) |
15     | WASB conda env | `wasb` (Python 3.8, torch 1.11.0+cu113) ? verified
`torch.cuda.is_available()==True` |
16     | Staging dir for videos | `~/clips` (avoids slow `/mnt/c` per-frame reads) |
17
18     GPU: RTX 2070 Super visible via `nvidia-smi` inside WSL (CUDA-on-WSL working).
19
20     Activate an env with:

```

```

21  ```bash
22  source ~/miniconda3/etc/profile.d/conda.sh
23  conda activate TrackNetV4    # or: conda activate wasb
24  ```
25
26  ## ?? Two upstream issues you must resolve before TrackNetV4 inference
27
28  1. **No pretrained weights are hosted.** The "Download" links in
29  `~/models/TrackNetV4/docs/RESULT.md` are placeholders (`(#)`). You will obtain
30  weights by **training** (`src/train.py`) on your annotated data ? which is the
31  plan. Until you have a `.h5`/`.keras` weights file, the segmenter's healthcheck
32  passes but `run_predict` will log an error that `weights_path` is unset.
33
34  2. **`predict.py` has a custom-objects bug.** `src/predict.py` references layers in
35  `custom_objects` that it never imports (`MotionIncorporationLayerV1`,
36  `MotionIncorporationLayerV2`, `CombineOutputs`, `MotionFramesInput`) ? it will
37  raise `NameError` on `load_model`. The imported layers are
38  `MotionPromptLayer`, `FusionLayerTypeA`, `FusionLayerTypeB`. Patch the
39  `custom_objects` dict in `main()` to match the layers your trained model
40  actually uses, e.g.:
41  ```python
42  model = load_model(model_weights_path, custom_objects={
43      'MotionPromptLayer': MotionPromptLayer,
44      'FusionLayerTypeA': FusionLayerTypeA,    # if you trained TrackNetV4_TypeA
45      'FusionLayerTypeB': FusionLayerTypeB,    # if you trained TrackNetV4_TypeB
46      'custom_loss': custom_loss,
47  })
48  ```
49
50  ## Wiring it into the indexer
51
52  The segmenter is `tracknet_hybrid` (`backend/pipeline/segmenters/tracknet_hybrid.py`),
53  backed by
54  `backend/pipeline/detectors/tracknet_runner.py`. Configure it in `config.json` under
55  `indexing.tracknet`:
56  ```jsonc
57  "tracknet": {
58      "wsl_distro": "Ubuntu",
59      "repo_dir": "~/models/TrackNetV4",
60      "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
61      "conda_env": "TrackNetV4",
62      "weights_path": "",           // <-- set to the WSL path of your trained weights
63      "queue_length": 5,
64      "stage_in_wsl": true,         // copy the video into ~/clips before inference
65      "wsl_stage_dir": "~/clips",
66      "timeout_sec": 1800,         // default 30 min; 0 = no timeout
67      "velocity_threshold": 0.02   // normalised shuttle displacement/sec to count as "in
68  flight"
69  }
70  ```
71
72  Then select it as the segmenter (e.g. set `indexing.default_segmenter` to
73  `tracknet_hybrid`, or pass it through whatever segmenter-selection your run uses).
74
75  ### Pipeline shape (mirrors `yolo_hybrid`)
76
77  1. **Phase 1 ? healthcheck:** activates the WSL env, confirms TF imports and a GPU
78  is visible. Fails fast with a clear message otherwise.
79
80  2. **Phase 2 ? TrackNet:** runs `predict.py` on the whole video, parses the
81  `Frame,Visibility,X,Y` CSV, and converts the shuttle trajectory into candidate
82  rally windows (visible + moving above `velocity_threshold`, grouped by
83  `dead_time_merge_gap`). Raw candidates are persisted for fine-tuning (same table
84  as YOLO candidates).
85
86  3. **Phase 3 ? AI handoff:** each candidate window is sent to the configured AI

```

```

84     provider for semantic boundary confirmation (identical to `yolo_hybrid`).
85 4. **Phase 4 ? DB:** confirmed segments written and linked back to their candidate.
86
87 ### Quick manual smoke test (inside WSL, once you have weights)
88
89 ```bash
90 source ~/miniconda3/etc/profile.d/conda.sh && conda activate TrackNetV4
91 cd ~/models/TrackNetV4/src
92 cp /mnt/c/Users/avidu/Projects/badminton-highlight-indexer/<clip>.mp4 ~/clips/
93 python predict.py --video_path ~/clips/<clip>.mp4 \
94     --model_weights <your_weights> --output_dir ~/clips/out
95 head ~/clips/out/<clip>_predict.csv # expect: Frame,Visibility,X,Y
96 ```
97
98 ## WASB (multi-sport ? and the faster path to a baseline)
99
100 **WASB ships pretrained weights**, including badminton, in
101 `~/models/WASB-SBDT/pretrained_weights/`:
102
103 - `wasb_badminton_best.pth.tar` (the WASB model)
104 - plus `tracknetv2_badminton_best.pth.tar`, `monotrack_badminton_best.pth.tar`,
105   `restracknetv2_badminton_best.pth.tar`, `ballseg_badminton_best.pth.tar`,
106   `deepball*_badminton_best.pth.tar` ? WASB re-implements all baselines.
107
108 This means **WASB can detect the shuttle on badminton video today**, whereas
109 TrackNetV4 has no hosted weights and must be trained first. If you want a working
110 trajectory baseline before your fine-tuning data is ready, WASB is the quicker
111 route.
112
113 Note: WASB's upstream repo documents dataset *evaluation*, not custom-video inference ?
114 so **this repo already ships the wrapper**: `backend/pipeline/detectors/wasb_infer.py`
115 (staged into WSL by `backend/pipeline/detectors/wasb_runner.py:WasbRunner`) runs the
116 detector + tracker on arbitrary video and emits the `Frame,Visibility,X,Y` CSV the
117 shared
118 windowing logic parses. The live `wasb_hybrid` segmenter uses it. Configure it via
119 `indexing.wasb` in `config.json`; see CODE_MAP §2.1/§2.2. (No need to build this ? it
120 exists.)
121
122

```

18. assistant (2026-06-19T16:57:40.994Z)

Now let me check the CODE_MAP claim about the single-target tracker explicitly, and verify whether the WASB-SBDT repo's LICENSE is checked out locally so I can directly verify "MIT" rather than relying on doc claims. Let me also confirm the tracker is genuinely "single-target" (single shuttle) as stated.

19. user (2026-06-19T16:57:43.001Z)

EXIT: not found if empty above

20. user (2026-06-19T16:57:44.598Z)

```

backend\pipeline\detectors\tracknet_runner.py:355:                "track_id_count": 1, # single
shuttle; kept for window-shape parity
backend\pipeline\detectors\wasb_infer.py:4:own building blocks (ImageDataset transform +
detector + online tracker) to run on
docs\QUALITY_ITERATIONS.md:591:> recall-starved corpus (WASB tracks one shuttle while several
courts rally ? served over-seg ratio < 1) the
docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:445: - Docs (both):
multiple targeted search/replace to excise every duplicated/misplaced dated Living Log block
(the post-#161 + T5 pre/ThisPR copies). Now each entry appears **once**, in the Living Log
(audit) or state transitions area (handoff), before any [Future] or in correct chronological
append position. Commands sections and References are clean/generic again. (Handoff also had its
"Next Actions" example left as-is since it is bootstrap illustration.)

```

21. assistant (2026-06-19T16:57:49.095Z)

The WASB-SBDT repo isn't checked out on the Windows side (it lives inside WSL at `~/models/WASB-SBDT`, not accessible from this Bash shell which is Git Bash on Windows, not WSL). Let me try the WSL path explicitly and also confirm the CODE_MAP's framing of the tracker as the single-target/online tracker, plus the `wasb\_hybrid` substrate.

22. user (2026-06-19T16:57:56.334Z)

```
Dockerfile
GET_STARTED.md
LICENSE.md
MODEL_ZOO.md
README.md
pretrained_weights
src
---LICENSE HEAD---
NO_LICENSE_FILE
---grep license type---
/home/avidullu/models/WASB-SBDT/README.md:![arXiv](https://img.shields.io/badge/arXiv-2311.05237-00ff00.svg)](https://arxiv.org/abs/2311.05237) [![License: MIT](https://img.shields.io/badge/License-MIT-yellow.svg)](https://opensource.org/licenses/MIT) ![test](https://img.shields.io/static/v1?label=By&message=Pytorch&color=red)
/home/avidullu/models/WASB-SBDT/README.md:https://github.com/nttcom/WASB-SBDT/assets/63090948/8889ef53-62c7-4c97-9b33-8bf386489ba1
/home/avidullu/models/WASB-SBDT/README.md:- [11/23/2023] [Our BMVC2023 proceeding](https://proceedings.bmvc2023.org/310/) is available! Thank you, BMVC2023 organizers!
/home/avidullu/models/WASB-SBDT/README.md:- [11/23/2023] Evaluation codes of DeepBall, DeepBall-Large and BallSeg are added!
/home/avidullu/models/WASB-SBDT/README.md:- [11/21/2023] Evaluation codes of TrackNetV2, ResTrackNetV2 and MonoTrack are added!
WSL_EXIT=0
```

23. assistant (2026-06-19T16:57:59.622Z)

The license file is `LICENSE.md`. Let me read it directly to confirm it's the MIT License text, and verify the tracker source in WASB-SBDT to confirm it's a single-target online tracker (which would inform whether a multi-target replacement is needed).

24. user (2026-06-19T16:58:01.909Z)

```
=== LICENSE.md ===
MIT License
```

Copyright (c) 2023 NTT Communications Corporation

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

```

=== trackers package ===
__init__.py
__pycache__
intra_frame_peak.py
online.py

=== build_tracker / tracker class head ===
/home/avidullu/models/WASB-SBDT/src/trackers/intra_frame_peak.py:3:class IntraFramePeakTracker:
/home/avidullu/models/WASB-SBDT/src/trackers/__init__.py:2:from .online import OnlineTracker
/home/avidullu/models/WASB-SBDT/src/trackers/__init__.py:6:     'online': OnlineTracker,
/home/avidullu/models/WASB-SBDT/src/trackers/__init__.py:9:def build_tracker(cfg):
/home/avidullu/models/WASB-SBDT/src/trackers/online.py:45:class OnlineTracker:

```

25. assistant (2026-06-19T16:58:04.986Z)

I now have direct confirmation of all three claims. Let me look at the OnlineTracker to confirm it's single-target (tracks a single shuttle) ? this is the crux of claim (b)'s "single-target" framing.

26. user (2026-06-19T16:58:07.680Z)

```

=== build_tracker body ===
from .intra_frame_peak import IntraFramePeakTracker
from .online import OnlineTracker

__tracker_factory = {
    'intra_frame_peak': IntraFramePeakTracker,
    'online': OnlineTracker,
}

def build_tracker(cfg):
    tracker_name = cfg['tracker']['name']
    if tracker_name not in __tracker_factory.keys():
        raise KeyError('unknown tracker: {}'.format(tracker_name))
    return __tracker_factory[tracker_name](cfg)

=== OnlineTracker.update head ===
5:         self._xy_dict    = {}
10:         self._xy_dict[fid]    = np.array([x,y])
22:         fids = list( self._xy_dict.keys() )
26:         if not fid in self._xy_dict.keys():
28:             return self._xy_dict[fid]
35:             xy1 = self._xy_dict[fid1]
36:             xy2 = self._xy_dict[fid2]
37:             xy3 = self._xy_dict[fid3]
84:     def update(self, frame_dets):
92:     def refresh(self):

```

27. user (2026-06-19T16:58:11.564Z)

```

class OnlineTracker:
    def __init__(self, cfg):
        self._max_disp = cfg['tracker']['max_disp']
        self._fid      = 0
        self._track     = Track()

    def _select_best(self, frame_dets):
        best_score = - np.Inf
        visi       = False
        x, y       = - np.Inf, - np.Inf

        xy_pred = None

```

```

        for det in frame_dets:
            score = det['score']
            if xy_pred is not None:
                qscore = self._compute_quality(xy_pred, det['xy'], self._track.xy(self._fid-1))
            score += qscore

        if score > best_score:
            best_score = score
            xy = det['xy']
            x,y = xy[0], xy[1]
            visi = True
        return x,y,visi,best_score

def _select_not_too_far(self, frame_dets):
    if (self._fid==0) or (not self._track.is_visible(self._fid-1)):
        return frame_dets

    frame_dets_ = []
    for det in frame_dets:
        if np.linalg.norm( det['xy'] - self._track.xy(self._fid-1) ) < self._max_disp:
            frame_dets_.append(det)
    return frame_dets_

def _compute_quality(self, xy1, xy2, xy3):
    return - np.linalg.norm( xy1-xy2 )

def update(self, frame_dets):
    frame_dets_ = self._select_not_too_far(frame_dets)
    x,y,visi,score = self._select_best(frame_dets_)
    self._track.add(self._fid, x, y, visi, score)

    self._fid += 1
    return {'x': x, 'y': y, 'visi': visi, 'score': score}

def refresh(self):
    self._fid = 0
    self._track = Track()

```

28. assistant (2026-06-19T16:58:19.892Z)

I now have complete, direct evidence for all three claims. The `OnlineTracker` is unambiguously single-target: it maintains a single `Track`, picks the single best detection per frame (`\_select\_best`), and gates candidates to those near the previous single position (`\_select\_not\_too\_far` via `max\_disp`). It tracks one shuttle.

Let me verify one more thing for claim (b): that the cached `det\_raw.jsonl` actually holds the full per-frame multi-candidate detections (the raw material a multi-target linker would need), independent of the tracker. I already saw this in `wasb\_infer.py` ?
`det\_by\_fid[fid].extend(plain)` accumulates ALL candidates per frame, and the tracker runs over them separately at line 533+. The cache stores `[x, y, score, scale]` per candidate. Let me confirm the trajectory_to_action_windows / parse functions truly have no torch/cv2/WSL dependency (pure Python).

The functions `parse\_trajectory\_csv`, `\_active\_frames`, `trajectory\_to\_action\_windows`, `\_trim\_inactive\_tail`, `\_split\_overlong\_groups`, `\_blob\_fallback\_chop`, `\_midpoint\_split\_index` use only `csv`, arithmetic, and `logging` ? no torch/cv2/numpy/WSL. Confirmed from the full read above.

I have everything needed. Let me record the structured verdict.

29. user (2026-06-19T16:58:45.432Z)

Structured output provided successfully

30. assistant (2026-06-19T16:58:59.695Z)

I have verified all three claims against the actual code and the WASB-SBDT source (inside WSL). All three are VERIFIED, with two important caveats the design doc must preserve.

Key evidence:

(a) VERIFIED ? The single-target tracker lives in the WASB-SBDT repo's `trackers` package, reached via `from trackers import build\_tracker` at `backend/pipeline/detectors/wasb\_infer.py:387`, used at lines 533-538. The module docstring (lines 1, 10-12) states it must run inside the WSL `wasb` conda env from the WASB-SBDT `src/` dir; `wasb\_runner.py:106-169` stages the wrapper into `~/models/WASB-SBDT/src/` and invokes it via `conda activate wasb`. I inspected the actual `~/models/WASB-SBDT/src/trackers/online.py::OnlineTracker` ? it is unambiguously single-target (one `Track`, `\_select\_best` picks the single highest-scored detection per frame, `\_select\_not\_too\_far` gates to the single previous position via `max\_disp`).

(b) VERIFIED ? The detector and tracker passes are already split, and the cached `det\_raw.jsonl` persists ALL per-frame candidate detections (`wasb\_infer.py:505`
`det\_by\_fid[fid].extend(plain)`, serialized as `[x,y,score,scale]`), independent of the tracker. The downstream `parse\_trajectory\_csv` / `trajectory\_to\_action\_windows` are pure-Python
`@staticmethod`s on `TrackNetRunner` (`tracknet\_runner.py:236-523`) with zero torch/cv2/WSL imports. A replacement multi-target linker can run as pure Python on Windows over those caches with no WSL/GPU/torch. One caveat: the doc should say "no WSL/GPU/torch" rather than "no third-party libraries" ? WASB's existing linking math uses numpy, which is a CPU pip dependency a custom linker may or may not use.

(c) VERIFIED ? WASB-SBDT code is MIT, confirmed at source: `~/models/WASB-SBDT/LICENSE.md` is the verbatim MIT License (Copyright (c) 2023 NTT Communications Corporation), corroborated by `docs/COMMERCIALIZATION.md:65`, `DECISIONS.md:244`, `CODE\_MAP.md:264`. A custom tracker authored in this repo is our own IP and clears the MIT/Apache/BSD-only guardrail in CLAUDE.md. Important caveat the doc must keep separate: the WASB *weights* (`wasb\_badminton\_best.pth.tar`) are academic-data-trained and NOT covered by the MIT code grant (`CODE\_MAP.md:270`, `DECISIONS.md:244`) ? replacing the tracker code does not change that weights-provenance question.

Relevant files (absolute paths):

- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\wasb\_infer.py`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\wasb\_runner.py`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\tracknet\_runner.py`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE\_MAP.md`,
`docs\COMMERCIALIZATION.md`, `docs\TRACKNET\_WSL\_SETUP.md`
- WSL source (external dep): `~/models/WASB-SBDT/LICENSE.md`, `~/models/WASB-SBDT/src/trackers/{\_\_init\_\_.py,online.py}`